Занятие 2: Окружение приложения, Docker и YAML

Контейнеризация, Docker Compose, конфигурация

Слайд 1: Состав окружения

Что такое окружение приложения?

- Операционная система
- Runtime (Python, Node.js, Java)
- Зависимости (библиотеки, пакеты)
- Конфигурация (переменные окружения)
- База данных
- Веб-сервер

Проблемы без контейнеризации:

- "Works on my machine"
- Разные версии зависимостей
- Сложное развертывание
- Конфликты между проектами

Слайд 2: Зачем нужны контейнеры

Преимущества Docker:

- Изоляция приложений
- Воспроизводимость окружения
- Упрощенное развертывание
- Быстрый старт
- Эффективное использование ресурсов

Ограничения Docker:

- Накладные ресурсы
- Сложность для сложных архитектур
- Безопасность (нужна правильная настройка)
- Хранение данных (volumes)

Контейнер vs Виртуальная машина

- VM: полноценная ОС, больше ресурсов
- Контейнер: разделяет ядро хоста, меньше ресурсов

Слайд 3: Образ и контейнер

Docker Image (Образ)

- Неизменяемый шаблон
- Содержит ОС, runtime, зависимости, код
- Хранится в registry (Docker Hub)
- Слоистая структура

Docker Container (Контейнер)

- Запущенный экземпляр образа
- Изолированное окружение
- Может быть остановлен, удален, пересоздан
- Изменения не сохраняются в образе

Docker Engine

- Управляет контейнерами
- Обеспечивает изоляцию
- Управляет сетью и хранилищем

Слайд 4: YAML: основы синтаксиса

Ключи и значения

```yaml
name: John
age: 30
city: Moscow
```

Отступы (важно!)

```yaml
person:
  name: John
  age: 30
  address:
    street: Main St
    number: 123
```

Списки

```yaml
users:
  - John
  - Jane
  - Bob
```

Многострочные строки

```yaml
description: |
  This is a
  multi-line
  string
```

Слайд 5: YAML vs JSON

YAML

```yaml
server:
  host: localhost
  port: 8080
  features:
    - auth
    - logging
```

JSON

```json
{
  "server": {
    "host": "localhost",
    "port": 8080,
    "features": ["auth", "logging"]
  }
}
```

Отличия:

- YAML более читаемый для человека
- JSON более строгий и универсальный
- YAML поддерживает комментарии
- YAML имеет больше синтаксического сахара

Слайд 6: Docker Compose: базовая структура

docker-compose.yml

```yaml
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
```

Основные секции:

- **version:** версия формата
- **services:** определение сервисов
- **volumes:** определение volumes
- **networks:** определение сетей

Слайд 7: Service: image и build

Использование готового образа

```yaml
services:
  web:
    image: nginx:latest
  db:
    image: postgres:15-alpine
```

Сборка из Dockerfile

```yaml
services:
  app:
    build: .
    # или
    build:
      context: ./app
      dockerfile: Dockerfile.prod
```

Dockerfile

```dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Слайд 8: Service: ports и environment

Проброс портов

```yaml
services:
  web:
    image: nginx
    ports:
      - "8080:80"      # хост:контейнер
      - "3000-3005:3000-3005"  # диапазон
```

Переменные окружения

```yaml
services:
  db:
    image: postgres
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mydb
    # или из файла
    env_file:
      - .env
```

Слайд 9: Service: volumes

Подключение папки хоста

```yaml
services:
  app:
    image: node
    volumes:
      - ./app:/app        # хост:контейнер
      - ./data:/data:ro   # read-only
```

Named volumes

```yaml
services:
  db:
    image: postgres
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Отличия:

- Bind mount: папка хоста → контейнер
- Named volume: управляется Docker

Слайд 10: Сеть контейнеров

Общая сеть по умолчанию

```
services:
  web:
    image: nginx
  db:
    image: postgres
```

- Сервисы могут обращаться друг к другу по имени
- web может подключиться к db как к хосту "db"

Кастомная сеть

```
services:
  web:
    networks:
      - frontend
  db:
    networks:
      - backend

networks:
  frontend:
  backend:
```

Слайд 11: Порты: хост vs контейнер

```
[Хост:8080] → [Контейнер:80]
```

Пример:

```yaml
services:
  nginx:
    image: nginx
    ports:
      - "8080:80"
```

- localhost:8080 → контейнер:80
- Контейнер недоступен извне без проброса
- Внутри сети Docker сервисы общаются по внутренним портам

Localhost в контейнере

- localhost внутри контейнера = сам контейнер
- Для доступа к другим сервисам используйте имя сервиса

Слайд 12: Управление контейнерами

Основные команды

```
# Запуск
docker-compose up
docker-compose up -d          # в фоне

# Остановка
docker-compose stop
docker-compose down            # остановить и удалить

# Пересоздание
docker-compose up -d --force-recreate

# Логи
docker-compose logs
docker-compose logs -f web     # follow
docker-compose logs --tail=100
```

Состояния контейнера

- Created
- Running
- Paused
- Exited
- Dead

Слайд 13: Диагностика

Просмотр состояния

```
docker-compose ps
docker ps -a
```

Логи

```
docker-compose logs
docker logs <container_id>
```

Вход в контейнер

```
docker-compose exec web bash
docker exec -it <container_id> sh
```

Проверка ресурсов

```
docker stats
```

Очистка

```
docker system prune        # удалить неиспользуемые
docker volume prune         # удалить неиспользуемые volumes
```

Слайд 14: Полный пример docker-compose.yml

```yaml
version: '3.8'

services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/mydb
    depends_on:
      - db
    volumes:
      - ./app:/app

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    volumes:
      - db_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  db_data:
```

Слайд 15: Практическое задание

Задание 1: Чтение конфигурации
Дан docker-compose.yml. Объясните:

- Какие сервисы запускаются?
- На каких портах они доступны?
- Какие переменные окружения заданы?
- Где хранятся данные БД?

Задание 2: Создание docker-compose.yml
Создайте конфигурацию для:

- Web-приложения на порту 3000
- PostgreSQL базы данных
- Redis для кэширования

Задание 3: Диагностика
Запустите контейнеры и:

- Проверьте их состояние
- Посмотрите логи
- Войдите в контейнер БД

Слайд 16: Вопрос для собеседования

Вопрос: Прочитайте конфигурацию и объясните запуск сервисов, доступ к ним и размещение данных.

Дано:

```yaml
services:
  app:
    build: .
    ports:
      - "8080:8000"
    volumes:
      - ./data:/app/data
    environment:
      - DB_HOST=db
  db:
    image: postgres
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Ожидаемый ответ:

- App собирается из Dockerfile
- Доступен на localhost:8080
- Папка ./data монтируется в /app/data
- БД доступна как хост "db"
- Данные БД в named volume pgdata